Supplement S4

Prior sensitivity analysis

Table of contents

1 Introduction 1

2 Setup 2
- 2.1 Load Packages . . . 2
- 2.2 Functions . . . 2
- 2.3 Load and Prepare Data . . . 3
- 2.4 Models . . . 3

3 Prior sets 4

4 Fit models 4

5 Convergence 5

6 Marginal F1-scores 5
- 6.1 Models without domains . . . 6
- 6.2 Domain models . . . 7
- 6.3 Relative to the interval width . . . 8

1 Introduction

Supplement to the paper "Confidence intervals for the F1-score in medical object detection: comparing bootstrap and Bayesian multilevel methods".

Code and data: https://github.com/ndevln/f1-score-ci-object-detection (data: `S1_Data_Pathologists.csv`, `S2_Data_MIDOG_Models.csv`).

The Bayesian binomial + Poisson models of Example 1 (pathologists vs ML model) and Example 2 (MIDOG challenge) use the brms default priors. This script refits all models with two alternative prior sets and compares the marginal pooled and grouped F1-scores (median and 95% QI, computed as in the examples).

Prior set	Intercepts	SDs (image, pathologist)	Correlation (pathologist)	Domain effects
default	brms default: student_t(3, μ, 2.5)	half-student_t(3, 0, 2.5)	lkj(1)	flat
tight	normal(0, 1.5)	half-normal(0, 1)	lkj(2)	normal(0, 1)
wide	normal(0, 10)	half-student_t(3, 0, 10)	lkj(1)	normal(0, 10)

The intercepts are on the logit scale (recall) and the log scale (FP count per image). The location μ of the default intercept prior is 0 for the recall part and the log median FP count for the FP part.

Models (8 models x 3 prior sets = 24 fits):

- Example 1: pathologist model (image and pathologist effects) and ML model.
- Example 2, Part A: the three ML models (image effects).
- Example 2, Part B: the three ML models with Domain as a fixed effect.

All fits use 4 chains with 1000 warmup and 4000 iterations (12,000 draws). The fits are saved in `fits_04/` and loaded on the next run (`brm(file = ...)`), so only the first run takes long (about 1-1.5 hours, mainly compiling the 24 Stan models).

2 Setup

2.1 Load Packages

```
# Seed (Article number of the Bertram et al. paper, as in Example 1)
set.seed(211067478)

# Data manipulation and visualisation
library(readr)
library(dplyr)
library(tidyr)
library(purrr)
library(ggplot2)
library(ggdist)

# Bayesian packages
library(brms)       # Fits the multilevel models via Stan
library(tidybayes)  # median_qi()/epred_draws() helpers for posterior summaries
```

2.2 Functions

The F1-score from the counts (paper eq. 3), as in the examples:

$$F_1 = \frac{2\,TP}{P + TP + FP}, \qquad P = TP + FN$$

`format_result()` prints a point estimate with its interval as "0.68 (0.67 - 0.70)".

```
calc_F1_score <- function(TP, FP, P) {
  2 * TP / (P + TP + FP)
}

format_result <- function(value, .lower, .upper) {
  fmt <- scales::label_number(accuracy = 0.01, trim = F)
  glue::glue("{fmt(value)} ({fmt(.lower)} - {fmt(.upper)})")
}
```

A prior set as brms priors. Every model only gets the priors of its parameters (brms rejects priors for parameters that are not in the model): the correlation only in the pathologist model, the domain effects only in the domain models. The default set returns `NULL`, i.e. the brms default priors.

```
make_priors <- function(intercept, sd, cor, b, rater, domain) {
  if (is.na(intercept)) return(NULL)
  priors <- set_prior(intercept, class = "Intercept", resp = c("TP", "FP")) +
    set_prior(sd, class = "sd", resp = c("TP", "FP"))
  if (rater) priors <- priors + set_prior(cor, class = "cor")
  if (domain) priors <- priors + set_prior(b, class = "b", resp = c("TP", "FP"))
  priors
}
```

Marginal pooled and grouped F1-score per posterior draw, as in the examples: expected counts of every image in **`newdata`** (with the fitted image effects; new pathologists get effects drawn from the fitted distribution), Bayesian bootstrap weights over the images (shared by all pathologists of an image), then the median and 95% QI. `by`: additional grouping, e.g. `"Domain"`.

```
calc_marginal_F1 <- function(fit, newdata, by = character()) {
  epred_draws(fit, newdata = newdata |> mutate(GroundTruth = 1),
              allow_new_levels = TRUE, sample_new_levels = "gaussian") |>
    ungroup() |>
    select(.draw, all_of(by), any_of("rater_ID"), slide, P, .category, .epred) |>
    pivot_wider(names_from = .category, values_from = .epred) |>
    mutate(TP = TP * P) |>   # TP part with trials = 1 is the recall
    # Bayesian bootstrap weights per image, shared by all pathologists of the image
    group_by(.draw, slide) |>
```

```
    mutate(weight = rexp(1)) |>
    group_by(.draw, across(all_of(by))) |>
    summarise(
      F1_pooled = calc_F1_score(sum(weight * TP), sum(weight * FP), sum(weight * P)),
      F1_grouped = weighted.mean(calc_F1_score(TP, FP, P), weight),
      .groups = "drop"
    ) |>
    pivot_longer(starts_with("F1_"), names_to = "averaging", names_prefix = "F1_") |>
    group_by(across(all_of(by)), averaging) |>
    median_qi(value) |>
    select(all_of(by), averaging, value, .lower, .upper)
}
```

2.3 Load and Prepare Data

As in the examples. In Example 2 the images without objects (paper "Case A") are excluded and the image column is renamed to `slide`, so both examples use the same model formula.

```
df_pathologists <- read_csv("S1_Data_Pathologists.csv", show_col_types = FALSE) |>
  mutate(rater_ID = as.factor(rater_ID), GroundTruth = TP + FN)

df_midog <- read_csv("S2_Data_MIDOG_Models.csv", show_col_types = FALSE) |>
  mutate(Domain = as.factor(Domain), GroundTruth = TP + FN) |>
  filter(GroundTruth > 0) |>
  rename(slide = Slide)
```

2.4 Models

The three binomial + Poisson models of the examples:

```
# Pathologist model (Example 1): image effects and correlated pathologist effects
bf_pathologists <-
  bf(TP | trials(GroundTruth) ~ 1 + (1|slide) + (1|r|rater_ID), family = binomial()) +
  bf(FP ~ 1 + (1|slide) + (1|r|rater_ID), family = poisson())

# ML model (Example 1 and Example 2, Part A): image effects
bf_ml <-
  bf(TP | trials(GroundTruth) ~ 1 + (1|slide), family = binomial()) +
  bf(FP ~ 1 + (1|slide), family = poisson())

# Domain model (Example 2, Part B): Domain as fixed effect and image effects
bf_domain <-
  bf(TP | trials(GroundTruth) ~ Domain + (1|slide), family = binomial()) +
  bf(FP ~ Domain + (1|slide), family = poisson())
```

The data of each model:

```
df_ex1_pathologists <- filter(df_pathologists, model == "Pathologists")
df_ex1_ml           <- filter(df_pathologists, model == "ML-Model")
df_ex2_model_1      <- filter(df_midog, model == "Model 1")
df_ex2_model_2      <- filter(df_midog, model == "Model 2")
df_ex2_model_3      <- filter(df_midog, model == "Model 3")
```

The 8 models, each with its formula and data. `rater` and `domain` mark the models with pathologist effects and domain effects (for the priors and the targets).

```
tribble(
  ~example,    ~model,           ~formula,        ~data,               ~rater, ~domain,
  "Example 1", "Pathologists",   bf_pathologists, df_ex1_pathologists, TRUE,   FALSE,
  "Example 1", "ML model",       bf_ml,           df_ex1_ml,           FALSE,  FALSE,
  "Example 2", "Model 1",        bf_ml,           df_ex2_model_1,      FALSE,  FALSE,
  "Example 2", "Model 2",        bf_ml,           df_ex2_model_2,      FALSE,  FALSE,
  "Example 2", "Model 3",        bf_ml,           df_ex2_model_3,      FALSE,  FALSE,
```

```
  "Example 2", "Model 1, domain", bf_domain,       df_ex2_model_1,      FALSE,  TRUE,
  "Example 2", "Model 2, domain", bf_domain,       df_ex2_model_2,      FALSE,  TRUE,
  "Example 2", "Model 3, domain", bf_domain,       df_ex2_model_3,      FALSE,  TRUE
) -> df_models

df_models |> select(example, model, rater, domain)
```

example	model	rater	domain
Example 1	Pathologists	TRUE	FALSE
Example 1	ML model	FALSE	FALSE
Example 2	Model 1	FALSE	FALSE
Example 2	Model 2	FALSE	FALSE
Example 2	Model 3	FALSE	FALSE
Example 2	Model 1, domain	FALSE	TRUE
Example 2	Model 2, domain	FALSE	TRUE
Example 2	Model 3, domain	FALSE	TRUE

3 Prior sets

`NA` = brms default prior.

```
tribble(
  ~prior_set, ~intercept,       ~sd,                   ~cor,     ~b,
  "default",  NA,               NA,                    NA,       NA,
  "tight",    "normal(0, 1.5)", "normal(0, 1)",        "lkj(2)", "normal(0, 1)",
  "wide",     "normal(0, 10)",  "student_t(3, 0, 10)", "lkj(1)", "normal(0, 10)"
) -> df_prior_sets

df_prior_sets
```

prior_set	intercept	sd	cor	b
default	NA	NA	NA	NA
tight	normal(0, 1.5)	normal(0, 1)	lkj(2)	normal(0, 1)
wide	normal(0, 10)	student_t(3, 0, 10)	lkj(1)	normal(0, 10)

4 Fit models

Every model with every prior set. The pathologist model uses the sampler settings of Example 1 (`adapt_delta = 0.95`, `max_treedepth = 12`), the other models the brms defaults. The fits are saved in `fits_04/` (`file_refit = "on_change"`: refit only if the data, formula or priors change).

```
dir.create("fits_04", showWarnings = FALSE)

df_models |>
  cross_join(df_prior_sets) |>
  mutate(
    fit = pmap(list(formula, data, rater, domain, intercept, sd, cor, b, model, prior_set),
               \(formula, data, rater, domain, intercept, sd, cor, b, model, prior_set)
      brm(formula,
          data = data,
          prior = make_priors(intercept, sd, cor, b, rater, domain),
          chains = 4, warmup = 1000, iter = 4000, cores = 4,
          control = if (rater) list(adapt_delta = 0.95, max_treedepth = 12),
          seed = 211067478,
          refresh = 0, # No sampling progress in the output for PDF
          file = file.path("fits_04", paste(model, prior_set, sep = "_")),
          file_refit = "on_change")
    )
  ) -> df_fits
```

5 Convergence

Max Rhat (should be < 1.01), min bulk and tail ESS (should be > 400) and the number of divergent transitions (should be 0) per fit.

```
df_fits |>
  mutate(diagnostics = map(fit, \(fit) {
    df_summary <- posterior::summarise_draws(as_draws_array(fit), "rhat", "ess_bulk", "ess_tail")
    tibble(
      max_rhat     = max(df_summary$rhat, na.rm = TRUE),
      min_ess_bulk = min(df_summary$ess_bulk, na.rm = TRUE),
      min_ess_tail = min(df_summary$ess_tail, na.rm = TRUE),
      divergences  = sum(subset(nuts_params(fit), Parameter == "divergent__")$Value)
    )
  })) |>
  select(example, model, prior_set, diagnostics) |>
  unnest(diagnostics)
```

example	model	prior_set	max_rhat	min_ess_bulk	min_ess_tail	divergences
Example 1	Pathologists	default	1.003748	876.2258	1726.600	0
Example 1	Pathologists	tight	1.002496	1299.6344	2271.849	0
Example 1	Pathologists	wide	1.004765	986.0151	1650.803	0
Example 1	ML model	default	1.003260	1799.9064	3506.988	0
Example 1	ML model	tight	1.001993	2110.2723	4045.142	0
Example 1	ML model	wide	1.002150	1493.3187	3266.304	0
Example 2	Model 1	default	1.003817	1805.0712	4538.147	0
Example 2	Model 1	tight	1.001873	2330.6116	4247.371	0
Example 2	Model 1	wide	1.001646	2212.2863	3659.606	0
Example 2	Model 2	default	1.002705	2027.3329	3948.876	0
Example 2	Model 2	tight	1.002037	2228.0356	3517.766	0
Example 2	Model 2	wide	1.001907	1944.1775	3750.914	0
Example 2	Model 3	default	1.002009	1975.1950	3204.489	0
Example 2	Model 3	tight	1.002185	1821.7483	3138.394	0
Example 2	Model 3	wide	1.001480	1753.7244	2892.266	0
Example 2	Model 1, domain	default	1.001590	2694.9015	5250.629	0
Example 2	Model 1, domain	tight	1.002235	2799.4243	4354.422	0
Example 2	Model 1, domain	wide	1.001939	2544.9740	4982.352	0
Example 2	Model 2, domain	default	1.002284	2733.3978	5118.924	0
Example 2	Model 2, domain	tight	1.001631	2965.0122	5261.095	0
Example 2	Model 2, domain	wide	1.001457	2536.2637	4947.174	0
Example 2	Model 3, domain	default	1.001422	2736.2145	4765.903	0
Example 2	Model 3, domain	tight	1.003019	2557.7179	5235.467	0
Example 2	Model 3, domain	wide	1.001806	2804.8216	4748.833	0

6 Marginal F1-scores

The targets of the examples:

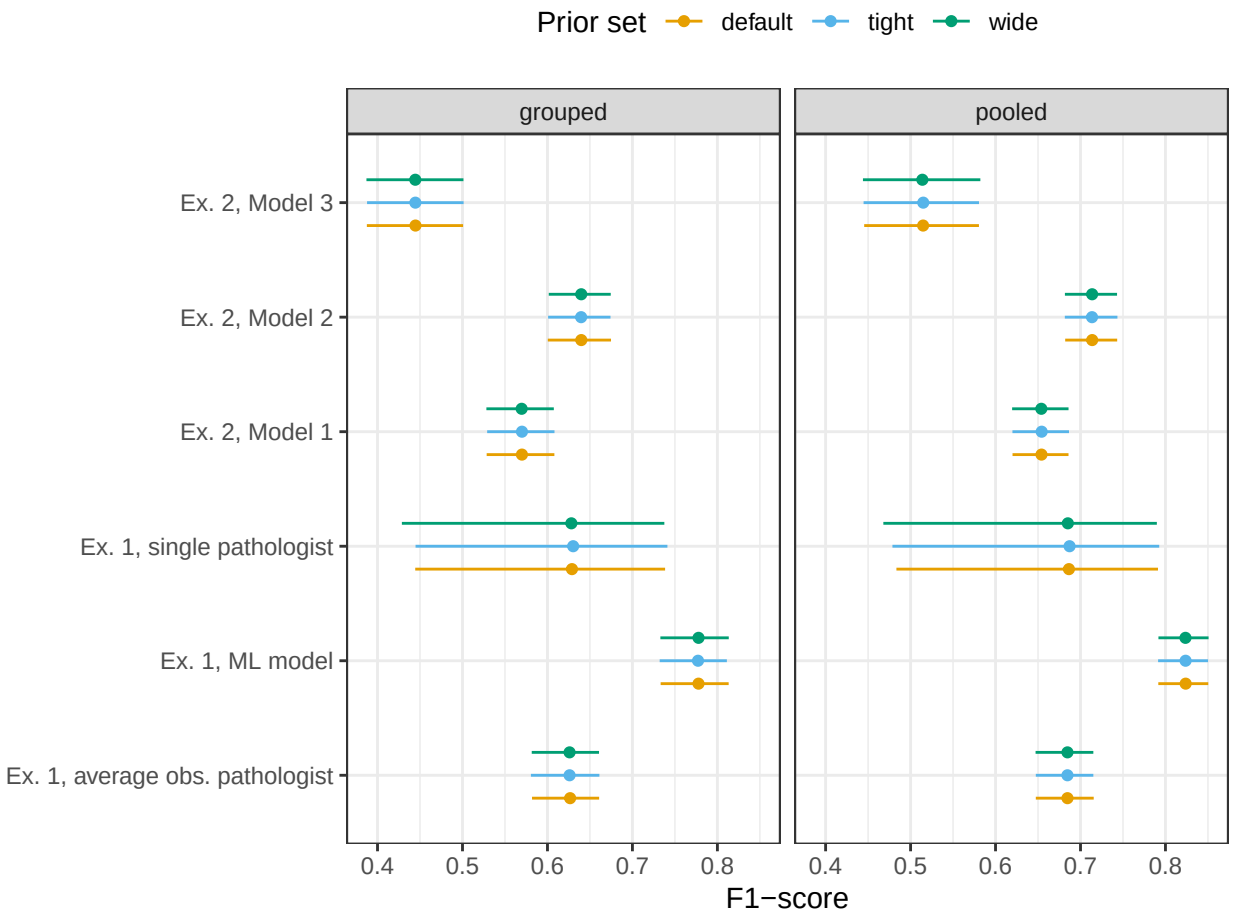
- Example 1, pathologist model: average of the observed pathologists (their fitted effects) and single pathologist (one new pathologist, prediction interval).
- Example 1 ML model and Example 2, Part A: the observed images of the ML model.
- Example 2, Part B: per domain.

```
df_fits |>
  mutate(estimates = pmap(list(fit, rater, domain), \(fit, rater, domain) {
    df_observed <- fit$data |> mutate(P = GroundTruth)
    bind_rows(
      calc_marginal_F1(fit, df_observed, by = if (domain) "Domain" else character()) |>
        mutate(target = if (rater) "average obs. pathologist" else "observed images"),
      if (rater) {
        calc_marginal_F1(fit, distinct(df_observed, slide, P) |>
                           mutate(rater_ID = "new_pathologist")) |>
          mutate(target = "single pathologist")
      }
```

```
    )
  })) |>
  select(example, model, domain, prior_set, estimates) |>
  unnest(estimates) |>
  # Short label for the tables and figures, e.g. "Ex. 1, average obs. pathologist"
  mutate(label = paste(sub("Example", "Ex.", example),
                       if_else(target == "observed images", model, target),
                       sep = ", ")) -> df_estimates
```

6.1 Models without domains

```
df_estimates |>
  filter(!domain) |>
  ggplot(aes(y = label, x = value, xmin = .lower, xmax = .upper, color = prior_set)) +
  geom_pointrange(position = position_dodge(width = 0.6), size = 0.2) +
  facet_wrap(~averaging, nrow = 1) +
  ggokabeito::scale_color_okabe_ito(name = "Prior set") +
  scale_x_continuous(name = "F1-score", breaks = (0:10) / 10) +
  ylab("") +
  theme_bw() +
  theme(legend.position = "top")
```



Supplement Figure S4.1: Marginal F1-score (median and 95% QI) of the example models with the default, tight and wide prior set.

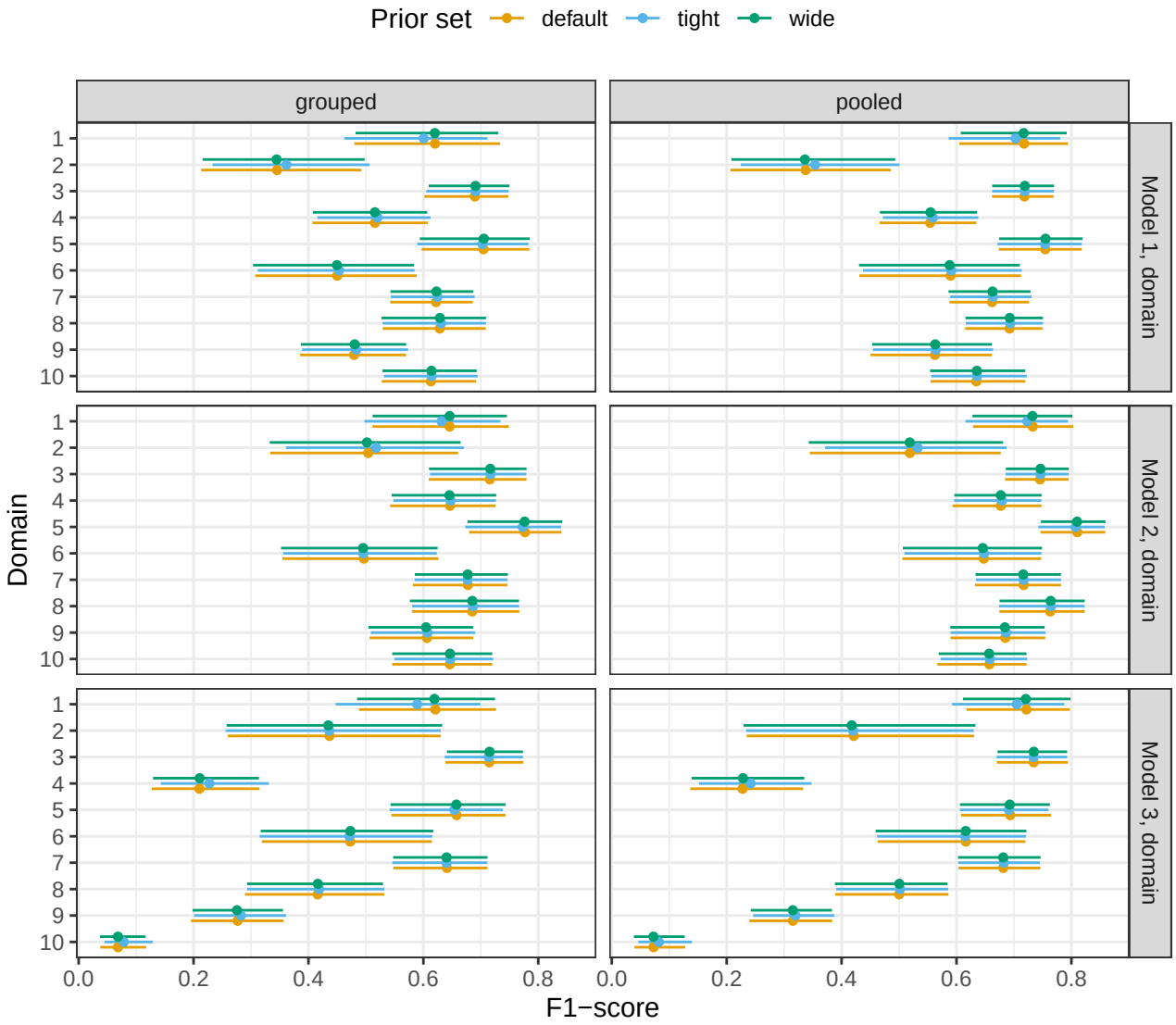
F1-score (median and 95% QI) per prior set:

```
df_estimates |>
  filter(!domain) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(label, averaging, prior_set, format) |>
  pivot_wider(names_from = prior_set, values_from = format)
```

label	averaging	default	tight	wide
Ex. 1, average obs. pathologist	grouped	0.63 (0.58 - 0.66)	0.63 (0.58 - 0.66)	0.63 (0.58 - 0.66)
Ex. 1, average obs. pathologist	pooled	0.68 (0.65 - 0.72)	0.68 (0.65 - 0.72)	0.68 (0.65 - 0.72)
Ex. 1, single pathologist	grouped	0.63 (0.44 - 0.74)	0.63 (0.44 - 0.74)	0.63 (0.43 - 0.74)
Ex. 1, single pathologist	pooled	0.69 (0.48 - 0.79)	0.69 (0.48 - 0.79)	0.69 (0.47 - 0.79)
Ex. 1, ML model	grouped	0.78 (0.73 - 0.81)	0.78 (0.73 - 0.81)	0.78 (0.73 - 0.81)
Ex. 1, ML model	pooled	0.82 (0.79 - 0.85)	0.82 (0.79 - 0.85)	0.82 (0.79 - 0.85)
Ex. 2, Model 1	grouped	0.57 (0.53 - 0.61)	0.57 (0.53 - 0.61)	0.57 (0.53 - 0.61)
Ex. 2, Model 1	pooled	0.65 (0.62 - 0.69)	0.65 (0.62 - 0.69)	0.65 (0.62 - 0.69)
Ex. 2, Model 2	grouped	0.64 (0.60 - 0.67)	0.64 (0.60 - 0.67)	0.64 (0.60 - 0.67)
Ex. 2, Model 2	pooled	0.71 (0.68 - 0.74)	0.71 (0.68 - 0.74)	0.71 (0.68 - 0.74)
Ex. 2, Model 3	grouped	0.44 (0.39 - 0.50)	0.44 (0.39 - 0.50)	0.44 (0.39 - 0.50)
Ex. 2, Model 3	pooled	0.51 (0.45 - 0.58)	0.51 (0.44 - 0.58)	0.51 (0.44 - 0.58)

6.2 Domain models

```
df_estimates |>
  filter(domain) |>
  ggplot(aes(y = Domain, x = value, xmin = .lower, xmax = .upper, color = prior_set)) +
  geom_pointrange(position = position_dodge(width = 0.6), size = 0.15) +
  facet_grid(model ~ averaging) +
  ggokabeito::scale_color_okabe_ito(name = "Prior set") +
  scale_x_continuous(name = "F1-score", breaks = (0:5) / 5) +
  scale_y_discrete(limits = rev) +
  theme_bw() +
  theme(legend.position = "top")
```



Supplement Figure S4.2: Marginal F1-score per domain (median and 95% QI) of the domain models with the default, tight and wide prior set.

6.3 Relative to the interval width

Change of the marginal F1-score with the tight and wide prior set compared with the default priors, relative to the uncertainty of the default estimate. Domain models (at the bottom): the largest change over the 10 domains.

```
df_estimates |>
  mutate(width = .upper - .lower) |>
  group_by(label, Domain, averaging) |>
  mutate(
    shift        = 100 * abs(value - value[prior_set == "default"]) / width[prior_set == "default"],
    width_change = 100 * (width / width[prior_set == "default"] - 1)
  ) |>
  filter(prior_set != "default") |>
  # Domain models: the largest change over the domains
  group_by(domain, label, averaging, prior_set) |>
  summarise(
    shift        = max(shift),
    width_change = width_change[which.max(abs(width_change))],
    .groups = "drop"
  ) |>
  mutate(column = factor(paste0(if_else(averaging == "pooled", "Pooled", "Grouped"), ", ", prior_set),
```

```
                          levels = c("Pooled, tight", "Pooled, wide", "Grouped, tight", "Grouped, wide"))) |>
  arrange(domain, label) -> df_sensitivity
```

Shift of the median in % of the width of the default 95% QI (e.g. 5: the median moves by 5% of the interval width):

```
df_sensitivity |>
  mutate(shift = round(shift, 1)) |>
  select(label, column, shift) |>
  pivot_wider(names_from = column, values_from = shift, names_sort = TRUE) |>
  rename(model = label)
```

model	Pooled, tight	Pooled, wide	Grouped, tight	Grouped, wide
Ex. 1, ML model	0.2	0.4	0.7	0.0
Ex. 1, average obs. pathologist	0.0	0.0	0.8	0.8
Ex. 1, single pathologist	0.2	0.4	0.5	0.2
Ex. 2, Model 1	0.3	0.3	0.1	0.4
Ex. 2, Model 2	0.3	0.1	0.3	0.1
Ex. 2, Model 3	0.1	0.7	0.1	0.2
Ex. 2, Model 1, domain	7.8	0.6	7.7	0.8
Ex. 2, Model 2, domain	5.4	0.7	5.9	0.8
Ex. 2, Model 3, domain	10.4	0.9	13.2	0.7

Change of the width of the 95% QI in % (negative: narrower than with the default priors):

```
df_sensitivity |>
  mutate(width_change = round(width_change, 1)) |>
  select(label, column, width_change) |>
  pivot_wider(names_from = column, values_from = width_change, names_sort = TRUE) |>
  rename(model = label)
```

model	Pooled, tight	Pooled, wide	Grouped, tight	Grouped, wide
Ex. 1, ML model	0.0	0.4	-1.2	0.5
Ex. 1, average obs. pathologist	-0.7	-0.2	2.1	0.3
Ex. 1, single pathologist	2.0	4.6	0.8	5.1
Ex. 2, Model 1	1.0	0.8	-0.6	-0.4
Ex. 2, Model 2	1.2	0.3	-1.3	-1.9
Ex. 2, Model 3	0.6	2.3	0.3	0.8
Ex. 2, Model 1, domain	2.5	2.9	3.0	-3.8
Ex. 2, Model 2, domain	-5.0	-2.0	-5.3	2.6
Ex. 2, Model 3, domain	8.5	4.1	5.7	-2.7